

wF-260701-DIS-420-FPGA_EXPLEO

TP02 Compteurs

Question 1

L'horloge du système est fixée à 100MHz. Combien de période faut-il compter pour attendre 2 secondes ? Combien de bits faut-il au minimum pour représenter cette valeur ?

La période d'une horloge de fréquence $f = 100 \text{ MHz}$ est 10 ns: $T = \frac{1}{f} = 10^{-8} \text{ s}$

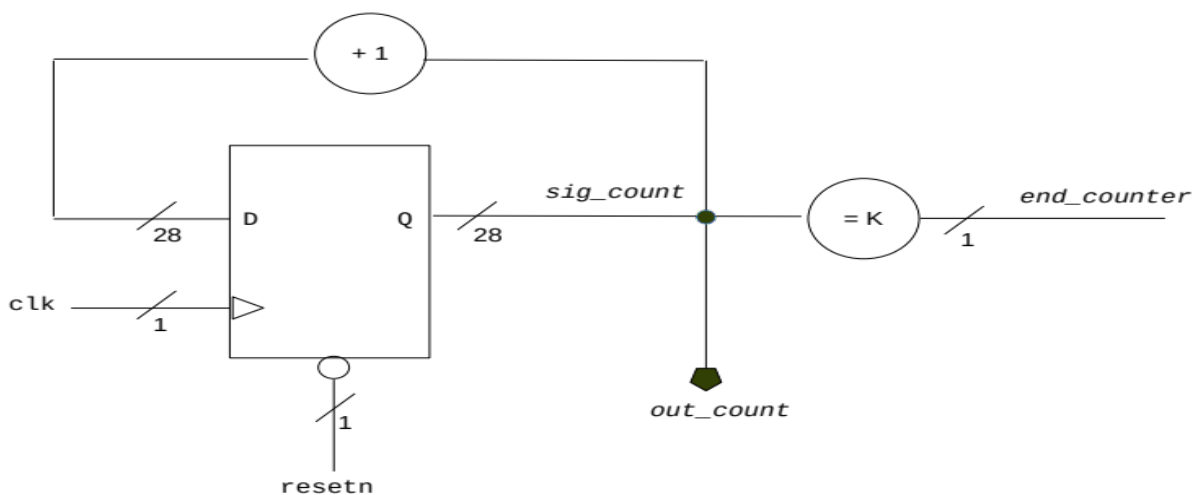
À un intervalle de temps $\Delta t = 2 \text{ s}$ correspond donc $K = \frac{\Delta t}{T} = 2 \cdot 10^8$ périodes.

$N = \lceil \log_2(K) \rceil$ bits sont *a priori* nécessaires pour représenter K , soit 28 bits.

Rem: N bits permettant de représenter les entiers non signés dans l'intervalle $[0, 2^N - 1]$, on vérifie également $K < 2^N$.

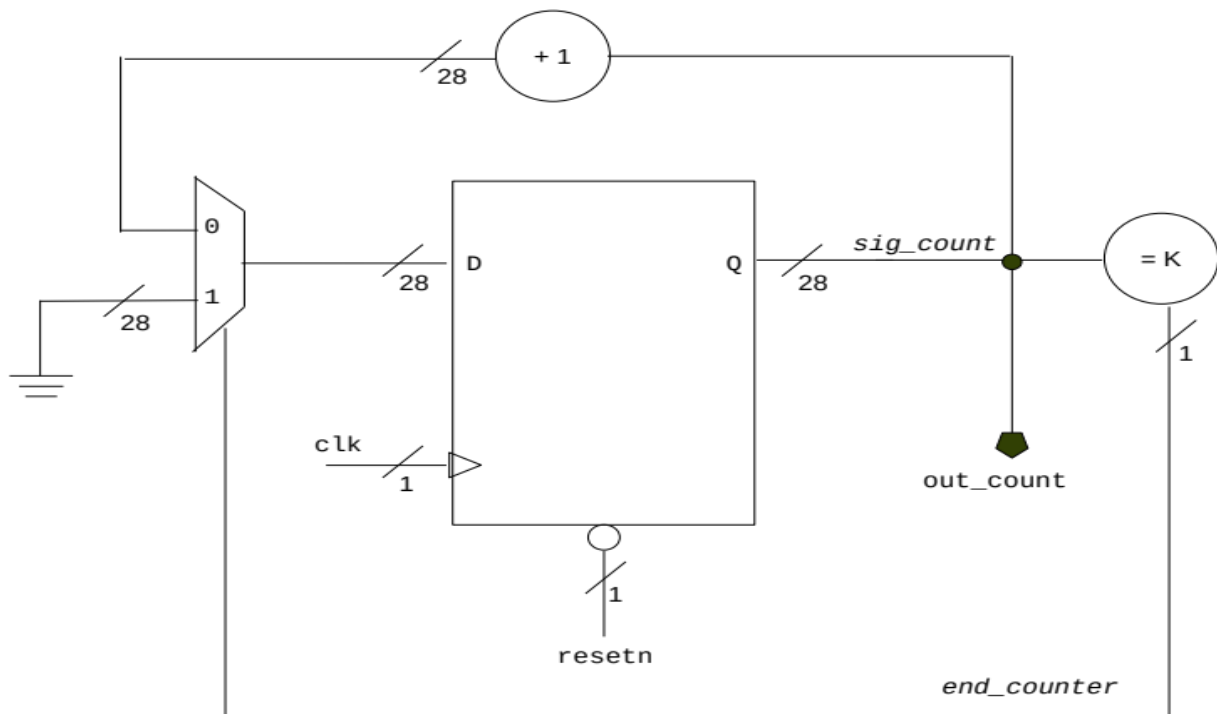
Question 2

Dessinez le schéma RTL de ce compteur. Si le compteur atteint la valeur calculée précédemment, un signal `end_counter` passe à 1, sinon `end_counter` vaut 0. N'oubliez pas de mettre sur chaque signal son nombre de bits. Commencez par réaliser une boucle d'incrémentation : +1 à chaque coup d'horloge.



Question 3

Ajoutez une condition pour que le compteur soit remis à 0 lorsqu'il a atteint la valeur souhaitée.



Question 4

Listez les signaux d'entrée, de sortie et les signaux internes de votre architecture.

L'unique port d'entrée est l'horloge `clk`.

L'unique port de sortie est `out_count`, valeur du compteur en nombre de fronts montants (*rising edges*).

Les signaux internes sont `end_counter` et `sig_count` (*previous data*).

Question 5

Ecrivez à présent le compteur en VHDL en suivant le schéma RTL, faites attention de bien faire correspondre les noms des signaux de votre code VHDL avec ceux de votre schéma RTL.

Voir `src/counter.vhd`.

Question 6

Ecrivez un fichier de testbench pour tester votre desig.

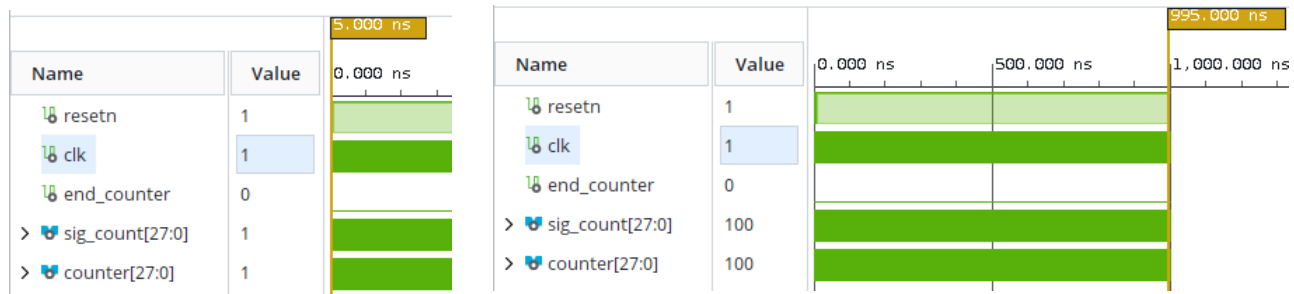
Voir src/tb_counter.vhd

Question 7

Lancez une simulation. Que devez-vous observer sur votre chronogramme pour vérifier que votre design est valide ?

En exécutant la simulation tb_counter.vhd pendant 1 ns (défaut) nous vérifions :

- Le compteur est incrémenté une première fois après 5 ns, c'est à dire après $\frac{T}{2}$:
l'horloge étant initialement à l'état LOW, le premier front montant apparaît après une demi-période.
- À l'instant $t=995\text{ ns}$, c'est à dire 99 périodes plus tard, nous mesurons bien 100 fronts montants



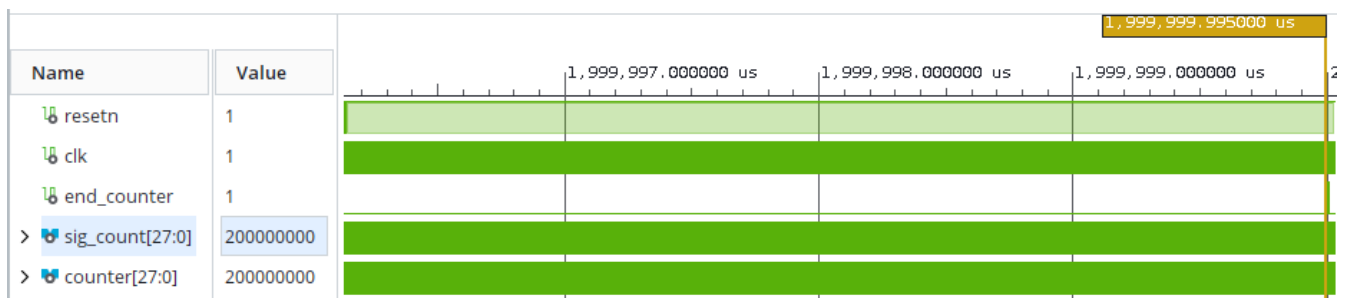
En exécutant la simulation avec $K=2000$, pendant 20.03 us, nous vérifions :

- Après 19995 ns, c'est à dire après une demi-période plus 1999 périodes, le compteur mesure bien 2000 fronts montants, et le signal interne `end_counter` montre que nous avons atteint le nombre maximum de périodes.
- Le compteur est remis à zéro sur le front montant suivant.

t (ns)	resetn	clk	end_counter	sig_count	out_count
t_0	0	0	U	0	U
5	1	1	0	1	1
15	1	1	0	2	2
...	...	...	...	...	...
19985	1	1	0	1999	1999
19995	1	1	1	2000	2000
20005	1	1	0	0	0
20015	1	1	0	1	1
20025	1	1	0	2	2



En exécuter la simulation avec $K=2.10^8$, pendant 2000000.03 us , nous vérifions de même qu'après 1999999.995 us , c'est à dire après une demi-période plus 199999999 périodes, le compteur mesure bien 2.10^8 fronts montants.

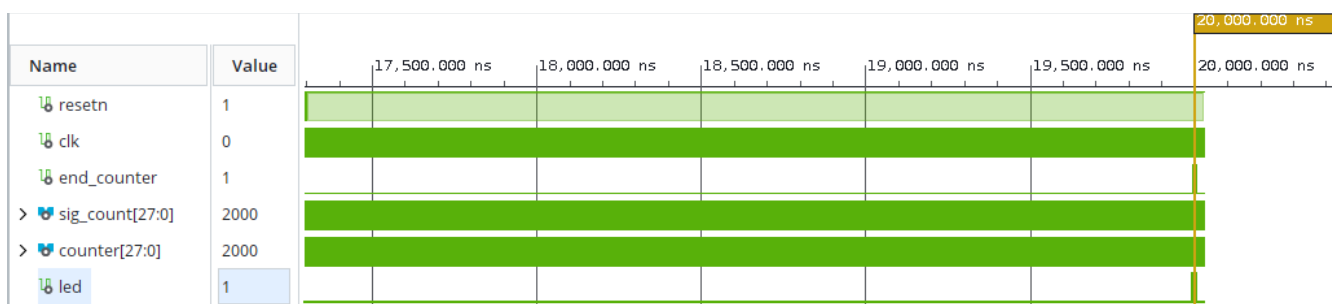


Question 8

Associez une LED avec le signal de teste d'arrêt du compteur. Pour cela, il faudra ajouter une sortie et la relier à une broche d'une LED dans le fichier de contrainte (.xdc). La LED sera alors allumée pendant seulement un coup d'horloge.

La LED est représentée par le port de sortie led0_b du compteur : voir counter_unit.vhd. Cette sortie est également ajoutée à la simulation src/tb_counter.vhd.

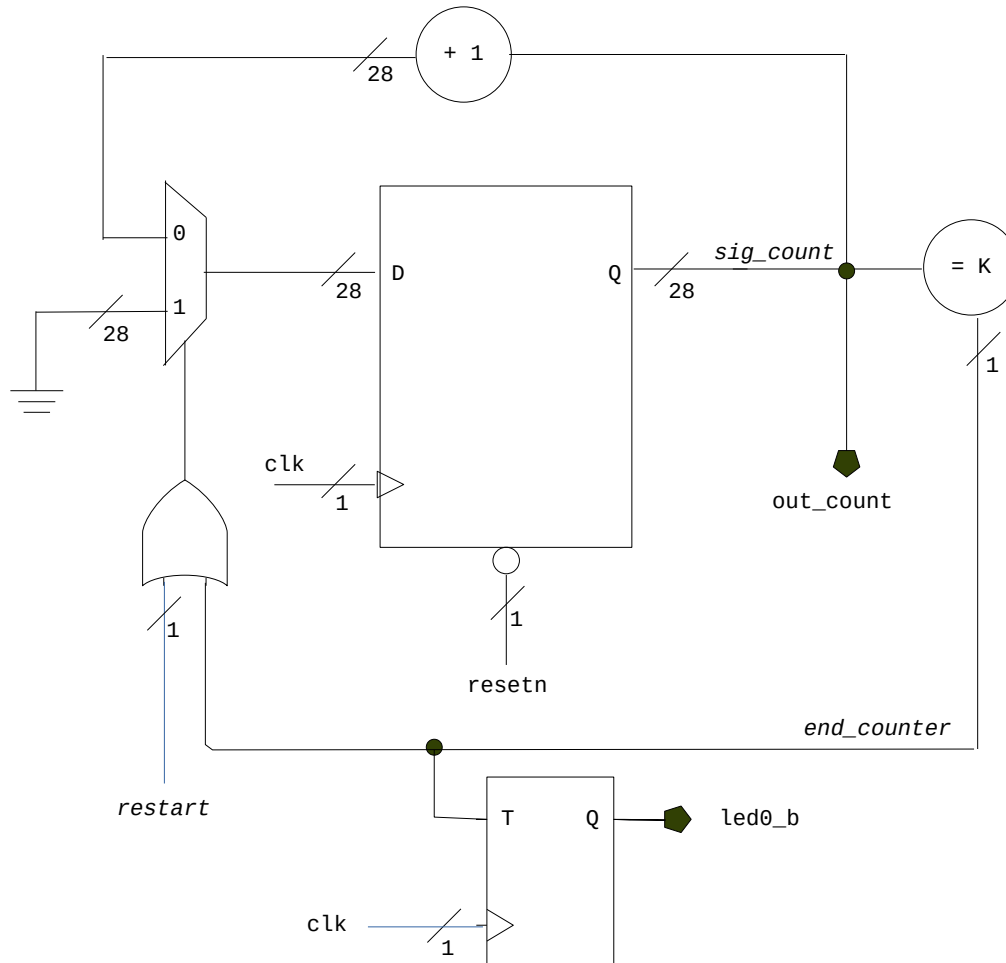
Exécuter cette simulation avec $K=2000$, pendant 20.03 us , nous permet de vérifier que la LED est bien ON lorsque le signal interne *end_counter* est HIGH.



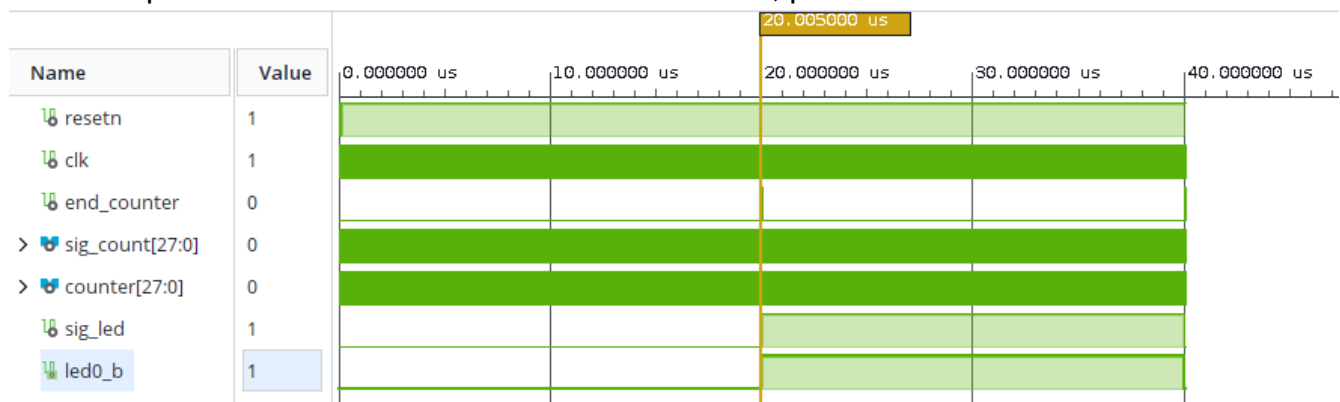
Question 9

Modifiez le schéma RTL du compteur pour ajouter une remise à 0 lorsqu'un signal restart est à 1. Ajoutez la logique nécessaire pour que la LED clignote telle que : allumée 2s, éteinte 2s.

Le principe est d'inverser l'état du port de sortie led0_b sur chaque événement end_counter, dont la périodicité est de 2 s.



Par exemple en exécutant la simulation avec $K=2000$, pendant 40.06 us :



Question 10

Faites les mises à jour nécessaires sur le code VHDL pour correspondre au nouveau schéma.

voir `src/counter_unit.vhd` et `src/tb_counter.vhd`.

Question 11

Associez la nouvelle entrée restart à un bouton.

Blabla

Question 12

Mettez à jour votre testbench puis vérifiez votre design avec une simulation. Quels sont les signaux que vous devez observer ?

Blabla

Question 13

Exécutez la synthèse puis ouvrez la schématique. Identifiez sur la schématique les différents éléments de votre architecture RTL.

Blabla

Question 14

Ouvrez le rapport de synthèse et relevez les ressources utilisées. Comparez vos résultats avec les résultats attendu selon votre architecture RTL.

Blabla

Question 15

Ouvrez le Set Up Debug. Placez des sondes sur les signaux à observer que vous avez défini à la question 12.

Blabla

Question 16

Lancez l'implémentation puis étudiez le rapport de timing (vérifiez les violations de set up et de hold et identifiez le chemin critique).

Blabla

Question 17

Générez le bitstream pour observer le système sur carte et le résultat de la ILA.

Blabla